

How a Decision App learns

The clause-memory mechanism, end to end.

This describes what happens between an officer disagreeing with a recommendation and that judgement changing the next recommendation. Every number quoted is the live default in the source; each is named so it can be checked rather than believed.

1. The problem

An SOP covers what someone wrote down. The cases that cost money are the others: every documented rule satisfied, every threshold cleared — and an experienced officer still says hold on. That officer is applying something no document contains, and when they leave it goes with them.

Retrieval over policy documents is solved. This mechanism is for the part the rulebook never covered.

Two things follow from that, and they shape everything below:

- **SOP always outranks a learned judgement.** Memory fills silence; it never overrides text.
 - **Learned judgement is evidence, never authority.** A clause changes what the agent argues, not what it is permitted to do.
-

2. Facets — the address of a case

Each app declares a closed vocabulary in its `case_signature`. A claim might resolve to:

Claim Type: hospitalisation	Claimed Band: 50000_250000	Fir: absent
Intimation Delay: gte_30	Surveyor: present	

These are **bands, not values** — 50000_250000, not ₹87,900; gte_30, not 45 days. A raw value is unique to one file and would match nothing again. Bands are what make two cases comparable.

This is the whole meaning of the line in the review panel: anything you teach here comes back on cases like these — and only those. The facets **are** "cases like these".

3. What each officer action records

action	reason	clause memory	clause statistics
Apply as proposed	not asked	nothing	fired +1 — counts as agreement
Apply with overrides	required	recorded as a correction	fired +1, blamed +1
Discard / reject	required	recorded as a correction	fired +1, blamed +1

A clean approve is discarded on purpose:

```
if not reason and not corrected_fields:
    return False # clean approve – nothing to learn
```

If agreement authored evidence, the system would learn most from the cases where it was already right, and forty routine approvals would become forty rules saying nothing.

But agreement is not wasted — see §8. It is the denominator that makes a clause's precision mean anything.

The reason box. Minimum 10 words, enforced; a soft warning past 20 words, not enforced; hard cap 500 characters. §9 explains why the ceiling exists.

4. The consolidation job

Clause formation is a background loop in `smart-app-service`, not a cron and not on the request path.

- Runs every **900 seconds** (`CONSOLIDATION_INTERVAL_SECONDS`).
- **Leader-elected**, so two workers cannot double-count officer support and defeat the promotion gate in §7.
- A bucket is only processed when it has **≥5 pending corrections** (`CONSOLIDATE_MIN_PENDING`) **or is ≥6 hours old** (`CONSOLIDATE_MAX_AGE_HOURS`).
- Pausable at runtime, with a status endpoint for inspection.

The officer's decision returns immediately and their reason is durable on the `DecisionRecord` regardless. Nothing about learning is allowed to slow down or break a decision.

In a demo: reject two cases and no clause appears. Five corrections, or the six-hour window, or a manual pass. This surprises people.

5. Clustering — finding corrections that say the same thing

Two officers will not use the same words:

"late intimation needs manager sign-off before settling" "cannot settle beyond the 30 day window without manager approval"

Corrections are grouped when they pass **two independent gates**.

Text similarity — Jaccard, ≥ 0.34 (`CLUSTER_SIMILARITY`). Strip stopwords and short words, then

$$\text{shared words} \div \text{total distinct words}$$

Facet compatibility — overlap coefficient, ≥ 0.5 (`CLUSTER_FACET_OVERLAP`), deliberately not Jaccard:

$$|A \cap B| \div \min(|A|, |B|)$$

Jaccard's denominator grows with every facet family an app declares, so two corrections about the same lesson that differ on incidental facets score lower the richer the signature is — a six-family app sharing three core facets came out $3/9 = 0.33$ and never clustered. Declaring more context must never make an app slower to learn. The overlap coefficient asks only "of the facets you could share, how many do you?" and is immune to each side's extras.

Why Jaccard for text. It is free, instant, deterministic, and its output never changes for the same input. An embedding model would handle synonyms better, but it does not fix the failure that matters here — negation — and it introduces one that does: a model upgrade silently re-partitions the rule store. Measured on this stack, `bge-m3` scored unrelated pairs 0.24–0.47 against related pairs 0.63–0.79, a usable gap of about 0.16 on general text, narrower still within one domain.

6. From cluster to clause

A cluster must clear three gates.

Size — at least 2 corrections (`MIN_CLUSTER_SIZE`), relaxed to 1 when the app has fewer distinct officers than the promotion gate, so a one-officer branch still learns.

Substance — the cluster's combined content tokens must reach **6** (`MIN_CONTENT_TOKENS`). Below that it is recorded as `insufficient_reason` and authors nothing. This is the real vagueness gate, and it counts tokens, not words.

Scope — which facets actually matter. A facet is kept only when its presence in the cluster is informative relative to how often it appears at all: **lift ≥ 1.3** (`LIFT_MIN`), over a sample of at least 20 (`MIN_BASE_RATE_SAMPLE`). If 90% of all claims carry `Surveyor: present`, that facet carries no information and is dropped. What remains is what makes the lesson specific.

Then, and only then, one LLM call. `author_clause_text()` is given the officers' quoted corrections and asked to phrase the lesson in a bounded number of words. That is the model's entire role.

Everything else is arithmetic:

decision	method
do these corrections group?	Jaccard $\geq$ 0.34, facet overlap $\geq$ 0.5
what does the clause apply to?	lift $\geq$ 1.3
does it match an existing clause?	similarity $\geq$ 0.5 (MATCH_SIMILARITY)
should two clauses merge?	$\geq$ 0.75 (MERGE_SIMILARITY)
is it trusted?	distinct officer count
is it working?	1 - <code>blamed/fired</code>

The model never decides whether a rule exists, what it applies to, or whether it is trusted. A bad generation produces awkward wording, not a wrong scope or a premature promotion.

It raises rather than falling back. A failed call leaves the corrections unconsumed so the next pass retries them. A placeholder string would become a permanent, unprovenanced rule.

7. Corroboration and status

A clause carries its provenance: which corrections formed it, which officers wrote them, when.

- **fewer than 3 distinct officers** → `candidate` — used, labelled provisional
- **3 or more** → `active`

Distinct matters. One person rejecting the same thing five times is one opinion repeated, not a team norm. At most 3 uncorroborated judgements (MAX_INDIVIDUAL_JUDGEMENTS) may be shown on a case: a case may consult a few individual opinions, never drown in twelve.

Statuses a clause can hold:

status	meaning
candidate	fewer than 3 officers; used, marked provisional
active	corroborated
dissented	disagreement share ≥ 0.34 (DISSENT_RATIO) — rendered as a notice, never as a rule
challenged	a supervisor stopped it pending adjudication
underperforming	measurably wrong too often — see §8
orphaned	scoped to a facet family the app no longer emits, so it can never fire
superseded / retired / quarantined / sop_conflict	withdrawn, replaced, or in conflict with an SOP

challenged exists for a specific failure: corroboration is a **headcount**, so three juniors sharing a misconception form a team judgement while the one person who knows better contributes one dissent in four — under the ratio, so nothing happens. It is deliberately **not** a seniority tier: weighting officers by rank would encode org hierarchy into an audit trail, and seniority is not correctness. It is a role-held stop that parks one clause and forces a named human to decide.

8. Retrieval, citation, and what agreement is for

Retrieval is set containment, not similarity. For a new case:

return clauses whose `scope_facets` are a **subset** of this case's facets

A clause scoped `{hospitalisation, gte_30}` fires on a hospitalisation claim intimated late. One scoped `{theft, fir_absent}` does not. Globally-scoped clauses always fire. Survivors are ranked and fitted to a word budget, and the injected clause ids are recorded on the run.

Note the split: **Jaccard is a write-time tool only.** Read time is exact.

Citation. Because provenance is kept, the agent can look up a clause and receive not just its text but the officers' original sentences and the cases behind them — deliberately, because the clause is one model's compression of several corrections and the originals often carry a condition the compression dropped.

What agreement is for. Every clause carries `fired_count` and `blamed_count`:

```
precision = (1.0 - blamed / fired) if fired >= MIN_FIRED_FOR_PRECISION else None
```

- **fired** — the clause was injected into a case
- **blamed** — the officer overruled the recommendation it shaped

So an approval is a silent vote that the clause was right. Below a precision of **0.7** (`PRECISION_FLOOR`) over at least **10** firings (`MIN_FIRED_FOR_PRECISION`), a clause is marked `underperforming` and **stops being applied**. Precision stays `None` until then, so an unproven clause is ranked on its prior rather than on a one-sample accident.

This is the evidence-based half of the seniority problem. Three juniors who agree can still form a judgement — but the moment the cases show it is wrong it stops being applied, and nobody has to outrank anyone.

8a. What happens as volume builds

The single most misread part of the mechanism: ten rejections do not make ten rules.

Ten rejections make about three clauses

The ten are **partitioned** by the two gates in §5. Officers reject for different reasons even on similar cases, so a realistic split is:

```
10 rejections
|- 4 about "late intimation needs manager approval" -> cluster
|- 3 about "discharge summary illegible"             -> cluster
|- 2 about "amount exceeds surveyor assessment"       -> cluster
\ - 1 one-off                                         -> no cluster
```

Each group clearing size ≥ 2 and ≥ 6 content tokens authors **one** clause.

The leftovers are not discarded. Corrections that do not cluster, and clusters too thin to author, are deliberately left unconsumed — "if an officer later writes a real reason for the same lesson, these ride along as corroboration". A rejection made today can help form a clause weeks later when a second officer says the same thing. Only clusters that actually author or reinforce a clause are marked consumed.

The next rejection reinforces; it does not rewrite

Every incoming cluster is checked against existing clauses **first**. On a match (similarity ≥ 0.5) the clause is reinforced — and **the text is never rewritten**. Only provenance, the officer list, `support_count`, `last_confirmed_at` and the match fingerprint widen.

That is the anti-dilution invariant, and it is the whole reason the previous design was removed: an LLM rewriting one summary per correction meant every correction degraded the ones before it, and no rule could ever be blamed for a bad recommendation.

So a clause does not grow in text. It grows in support.

Many opinions on the same facets

Four outcomes, and the system never averages them:

situation	outcome
same lesson, different words (≥ 0.5)	reinforced — one clause, more officers
genuinely different lesson, same facets	a second clause; both fire
near-identical text (≥ 0.75), one scope a subset of the other	merged — the more general survives, <code>merged_from</code> keeps it reversible
same field, opposing directions	both $\rightarrow$ <code>dissented</code> ; suppressed, shown as a disagreement notice

Contradiction detection is deliberately narrow: two rules about one field on overlapping cases are usually complementary, and treating that as a clash would silence real knowledge.

Three counters, three different jobs

- `support_count` — distinct officers; crossing 3 promotes candidate $\rightarrow$ active
- `fired / blamed` — precision; below 0.7 over ≥ 10 firings $\rightarrow$ `underperforming`, stops firing
- **dissent share** — $\geq 0.34 \rightarrow$ `dissented`

Steady state for a busy facet is a handful of clauses, each with several officers behind it, each carrying a live precision score, and any real disagreement parked rather than averaged away.

8b. Yes — several clauses inject on one case

`scope_facets  $\subseteq$  case_facets` is a filter, not a picker. Every clause that matches is a candidate, then ranked and fitted into a **1000-word budget**.

Clauses are split into three tiers first:

tier	statuses	cap
team judgements	<code>active</code>	budget only
individual judgements	<code>candidate</code>	3 (<code>MAX_INDIVIDUAL_JUDGEMENTS</code>)
notices	<code>dissented</code> , <code>sop_conflict</code>	2 dissent lines

Tier is the primary key. Every team judgement outranks every individual one, however specific the individual is — corroboration beats precision of scope.

Within a tier the sort is (scope_size DESC, score DESC) — **specificity first**, and that ordering is an n-gram backoff. A thin (theft ∧ photo ∧ us ∧ >25k) cell falls through to (theft ∧ photo), then (theft), with no special-casing and no cold-start cliff.

The tie-break score within a tier is

$$W_SUPPORT \cdot \log(1+support) + W_PRECISION \cdot precision + W_RECENCY \cdot recency$$

so a well-supported, accurate, recently-confirmed clause outranks a stale one at the same specificity. An unproven clause uses a prior rather than a one-sample accident.

Finally, **dedupe keeps the most specific survivor per (reason_code, contested_fields)** — a general clause is redundant once a narrower one on the same lesson fires. Across tiers this means a team judgement silently shadows an individual one on the same lesson, which is usually right: by the time a team judgement exists, the individual's evidence is normally already inside it.

The injected clause ids are recorded on the run, which is what later makes fired and blamed countable.

9. Known limits

Stated plainly, because a mechanism whose limits are hidden gets trusted where it should not be.

Jaccard is length-sensitive. Its denominator is the union of words, so long prose adds mostly non-shared tokens. Measured on the working corpus, of 15 pairs that cluster today: at 20 words 7 survive, at 25 only 1, past 30 none. A careful 60-word correction is recorded, never clusters, and never becomes a clause — which is why the reason box warns past 20 words. Terse is not vague: an 11-word correction produced a real clause.

Neither Jaccard nor embeddings handle negation. "needs manager approval" and "does not need manager approval" score as near-identical under both. A polarity guard — refusing to cluster corrections that disagree on polarity — is the cheapest real fix and is not yet built.

Agreement is a weak signal. An officer who approves because the recommendation was right and one who approves because they were rushing both produce fired +1. This is tolerable while a human dispositions every case; it would become dangerous if approvals were ever auto-generated, because the denominator would stop meaning "a person looked at this".

The thresholds are provisional. They were tuned against 18 seeded demo corrections with zero real officers behind them. insufficient_reason is recorded on every too-thin correction; that counter is the detector — wait for it to climb on real usage before re-tuning.

The durable fix for §5 and §9 is normalisation, not a better metric. Extracting a canonical gist — condition → action → polarity — and clustering on that removes length-sensitivity, synonym variance, and negation in one move, and makes the choice of similarity function largely moot.

10. Where this lives

concern	file
clustering, scope, authoring, the job	<code>smart-app-service/consolidation.py</code>
storage, retrieval, status, precision	<code>smart-app-service/clause_store.py</code>
the fold from an officer action	<code>smart-app-service/analysis_rubrics.py</code>
facets and the case signature	<code>smart-app-service/case_signature.py</code>
the scheduler	<code>smart-app-service/main.py</code>

Every threshold named here is environment-overridable; the constants carry the reasoning for their value in comments beside them.